

LayerNorm vs RMSNorm

Reading the stream at the right scale · video 2:38

This lesson answers one question: how does each block read a residual stream that keeps growing? Through a **normalization**. GPT-2 uses **LayerNorm** (subtract the mean, divide by the spread, scale and shift); Llama, Mistral and Qwen use **RMSNorm** (divide by the root mean square, scale). Both are computed by hand and match the real models. In a tiny GPT they give the same loss; without any normalization, training blows up at a higher learning rate.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Why normalize	▶ 0:08-0:40	2	☐
2. LayerNorm and RMSNorm, by hand	▶ 0:40-1:24	4	☐
3. Does it matter?	▶ 1:24-1:55	2	☐
4. Why modern models use RMSNorm	▶ 1:55-2:28	1	☐

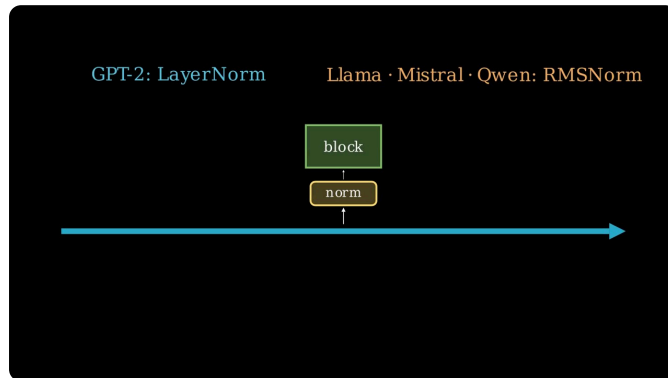
Before you start: Deep Dive episode 13 (the residual stream) and Foundations (mean, standard deviation). Code: `code/d14_norms` (GPT-2 small and Qwen2.5-0.5B, about 1.5 GB, plus six tiny models, about 25 minutes on a CPU).

CONCEPT 1 OF 4

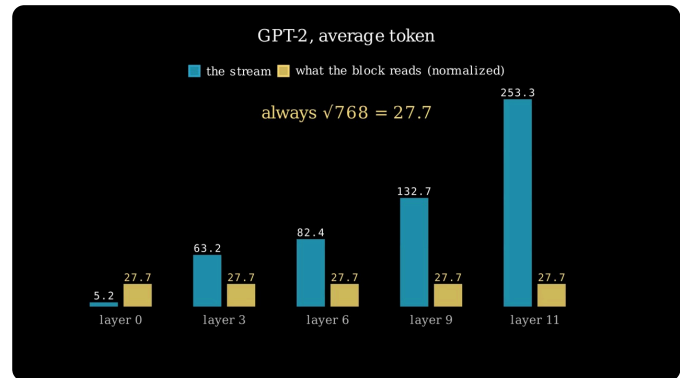
Why normalize

▶ Video 0:08–0:40

Every block reads the residual stream through a normalization. In GPT-2 the stream grows from an average size of **5.2** (layer 0) to **253.3** (layer 11). After normalization, every block reads a vector of exactly the same size, **27.7** = $\sqrt{768}$, at every layer: the block's weights see inputs on a fixed scale, however big the stream has become.



▶ 0:19 Each block reads the stream through a norm: LayerNorm in GPT-2, RMSNorm in Llama, Mistral and Qwen.



▶ 0:38 The stream grows from 5.2 to 253.3; what each block reads is always $\sqrt{768} = 27.7$.

KEY IDEA

Normalizing to unit spread makes a d -number vector's length $\sqrt{d}$, at every layer.

Check 1: Why normalize

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

Qwen2.5-0.5B's stream has 896 numbers. What is the length of a normalized vector (before the learned scale)?

1.2 TRUE OR FALSE

“Normalization changes the residual stream itself.”

True ☐ False ☐

CONCEPT 2 OF 4

LayerNorm and RMSNorm, by hand

▶ Video 0:40-1:24

LayerNorm. The token “ sat” entering GPT-2's layer 6 is 768 numbers with mean **0.088** and standard deviation **3.465**. Subtract the mean, divide by the standard deviation, then multiply by a learned scale (γ) and add a learned shift (β), 768 numbers each. The hand computation matches GPT-2's LayerNorm to within 4.8×10^{-7} .

RMSNorm skips the mean: divide by $\sqrt{(\text{mean}(x^2) + \epsilon)}$, multiply by a learned scale. No centering, no shift. In Qwen2.5-0.5B (layer 6, 896 numbers) the stream's mean is 0.022, almost zero anyway; its root mean square is 0.445. Again the hand version matches the model's to 4.8×10^{-7} .

LayerNorm, by hand: “ sat” entering GPT-2 layer 6

```
[ -0.316, +3.483, -0.591, +0.704, ... ]  
                        768 numbers  
  
mean 0.088 · standard deviation 3.465  
  
      (x - mean) ÷ std  
[ -0.117, +0.980, -0.196, +0.178, ... ]  
  
× learned scale + learned shift   (768 + 768 numbers)  
  
matches GPT-2's LayerNorm to  $4.8 \times 10^{-7}$ 
```

RMSNorm: skip the mean

```
x ÷ √(mean(x²) + ε)  
  × learned scale  
no centering, no shift  
  
Qwen2.5-0.5B, layer 6 (896 numbers): mean 0.022, RMS 0.445  
  
matches Qwen's RMSNorm to  $4.8 \times 10^{-7}$ 
```

▶ 1:04 LayerNorm on “ sat” entering GPT-2 layer 6: mean 0.088, std 3.465; normalize, then learned scale and shift.

▶ 1:22 RMSNorm: divide by the root mean square and scale; in Qwen2.5 the mean is 0.022 anyway.

KEY IDEA

LayerNorm = center + rescale + scale + shift; RMSNorm = rescale + scale.

Check 2: LayerNorm and RMSNorm, by hand

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

LayerNorm (no learned scale or shift) of [1, 2, 3, 6]?

.....

2.2 CALCULATION

RMSNorm (no learned scale) of [1, 2, 3, 6]?

.....

2.3 SHORT ANSWER

Add 10 to every number: [11, 12, 13, 16]. Which normalization gives the same output as before, and why?

.....

2.4 CALCULATION

How many learned numbers do GPT-2 small's 25 LayerNorms have in total?

.....

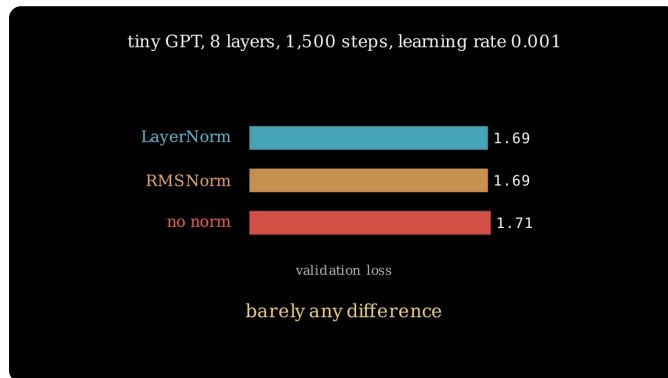
CONCEPT 3 OF 4

Does it matter?

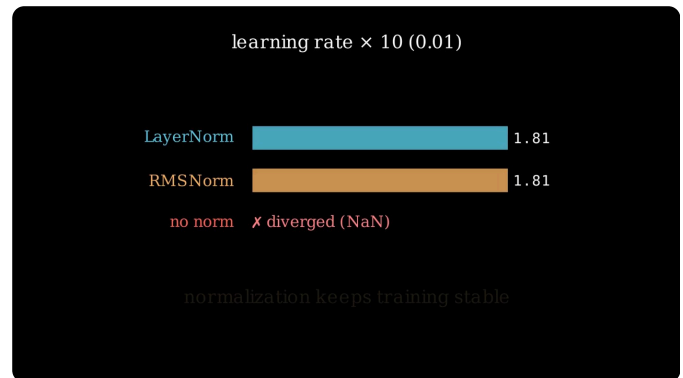
▶ Video 1:24-1:55

The same 8-layer tiny GPT, trained 1,500 steps three ways. At learning rate 0.001: LayerNorm **1.69**, RMSNorm **1.69**, no normalization **1.71**. Barely any difference.

Raise the learning rate ten times (0.01): LayerNorm and RMSNorm both reach **1.81**. Without normalization, the loss becomes NaN (“not a number”): the training blows up. Normalization is what keeps it stable.



▶ 1:39 Learning rate 0.001: LayerNorm 1.69, RMSNorm 1.69, no norm 1.71.



▶ 1:54 Learning rate 0.01: LayerNorm and RMSNorm 1.81; without a norm, the loss becomes NaN.

KEY IDEA

The choice of norm barely matters; having one matters a lot, especially for stability.

Check 3: Does it matter?

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

What happened without normalization at learning rate 0.01?

- A. Loss 1.71
- B. Same as LayerNorm
- C. The loss became NaN
- D. Training was faster

3.2 TRUE OR FALSE

“RMSNorm gave a worse loss than LayerNorm in the episode's experiment.”

True ☐ False ☐

CONCEPT 4 OF 4

Why modern models use RMSNorm

▶ Video 1:55-2:28

RMSNorm is simpler: one statistic instead of two, and no shift. Centering turned out to matter little. Speed, though, depends on the implementation: on this CPU, PyTorch's built-in LayerNorm took **15 ms** and its RMSNorm **46 ms** for 4,096 vectors of 4,096 numbers (a second run: 12 and 35 ms; times vary, the ratio of about 3 stays). With equally optimized code, RMSNorm does slightly less work.

	LayerNorm	RMSNorm
statistics	mean + spread	root mean square
learned	scale + shift	scale only
this CPU (PyTorch)	15 ms	46 ms

centering turned out to matter little

equally optimized: RMSNorm does slightly less work

```
1 class RMSNorm(nn.Module):
2     def forward(self, x):
3         rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
4         return x * rms * self.weight
5
6 # LayerNorm, for comparison:
7 # (x - mean(x)) / sqrt(var(x) + eps) * weight + bias
```

▶ 2:16 LayerNorm: two statistics, scale and shift. RMSNorm: one statistic, scale only. Speed depends on the implementation.

▶ 2:26 RMSNorm in code: multiply by the reciprocal root mean square, then by the learned weight.

KEY IDEA

RMSNorm keeps what matters (a fixed scale) and drops what doesn't (centering and shift); measure speed rather than assume it.

Check 4: Why modern models use RMSNorm

Answer all 1 questions before looking at the answers. Pass mark: all correct.

4.1 HANDS-ON CODE

Try it yourself. Check that the episode's `RMSNorm` class matches PyTorch's built-in one.

```
x = torch.randn(8, 128)
mine, theirs = RMSNorm(128), nn.RMSNorm(128, eps=1e-6)
print((mine(x) - theirs(x)).abs().max())
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Why normalize · rewatch ▶ 0:08-0:40

1.1 About 29.93. $\sqrt{896} \approx 29.93$, as $\sqrt{768} = 27.7$ for GPT-2.

1.2 False. Only the block's input is normalized; the stream keeps growing ($5.2 \rightarrow 253.3$) and the block's output is added to it unnormalized.

2. LayerNorm and RMSNorm, by hand · rewatch ▶ 0:40-1:24

2.1 About [-1.069, -0.535, 0, 1.604]. Mean 3, variance $(4 + 1 + 0 + 9) / 4 = 3.5$, standard deviation 1.871.

2.2 About [0.283, 0.566, 0.849, 1.697]. $\text{RMS} = \sqrt{((1 + 4 + 9 + 36) / 4)} = \sqrt{12.5} \approx 3.536$.

2.3 LayerNorm: it subtracts the mean, so a constant shift disappears ([-1.069, -0.535, 0, 1.604] again). RMSNorm changes: [0.838, 0.914, 0.990, 1.218]. Centering is exactly what RMSNorm leaves out.

2.4 38,400. 2 per layer $\times$ 12 + the final one = 25, each with 768 + 768.

3. Does it matter? · rewatch ▶ 1:24-1:55

3.1 C. The activations grew without bound and the loss became “not a number”.

3.2 False. 1.69 vs 1.69 at 0.001, and 1.81 vs 1.81 at 0.01.

4. Why modern models use RMSNorm · rewatch ▶ 1:55-2:28

4.1 0 (or about 10^{-7}): the same formula. Both compute $x / \sqrt{(\text{mean}(x^2) + \epsilon)}$ $\times$ weight, with weights initialized to 1.